

MoldAI Engine 项目深度分析文档（面试准备用）

目录

- [1. 项目概览与定位](#)
- [2. 整体架构设计](#)
- [3. 核心模块详解](#)
- [4. 关键设计模式与技术决策](#)
- [5. 进程间通信与并发模型](#)
- [6. HTTP 通信与 API 集成](#)
- [7. 积分/计费系统](#)
- [8. 数据序列化策略](#)
- [9. 摄影测量算法管线](#)
- [10. 生成式 AI 任务集成](#)
- [11. 构建系统与依赖管理](#)
- [12. 容错与异常处理](#)
- [13. 启动诊断机制](#)
- [14. 代码质量与改进方向](#)
- [15. 面试高频问题 Q&A](#)

1. 项目概览与定位

1.1 项目是什么？

MoldAI Engine 是一个 C++17 编写的桌面端摄影测量与 3D 重建引擎，运行在 Windows 平台上。它是 MoldAI 产品的核心计算后端，负责：

- 空中三角测量 (Aerial Triangulation, AT)**：从航拍/地面照片中恢复相机姿态和稀疏点云
- 三维重建 (3D Reconstruction)**：从多视角图像生成稠密点云和网格模型
- 生成式 AI 3D 任务**：通过调用远程 AI 服务，从文本描述或图像生成 3D 模型 (Text-to-3D / Image-to-3D)
- 积分计费系统**：对上述所有计算任务进行积分预估、冻结、结算

1.2 产品形态

项目编译为 **三个独立可执行文件** + **三个共享库**：

组件	类型	角色
MoldiaNode.exe	可执行文件	引擎调度守护进程——轮询任务队列、调度执行、管理生命周期
MoldiaTask.exe	可执行文件	算法工作进程——执行单个 AT/重建任务步骤后退出

组件	类型	角色
MokCopy.exe	可执行文件	OTA 更新工具——打包上传/下载版本更新
MoldiaData.dll	动态库	核心算法库——AT、重建、坐标变换、数据模型
MoldiaOSGEditor.dll	动态库	3D 编辑器——基于 OpenSceneGraph 的可视化编辑
3DViewer.lib	静态库	3D 查看器——OSG+Qt 的模型查看组件

1.3 技术栈

- **语言**：C++17 (MSVC 编译器, `/utf-8` 标志)
- **UI 框架**：Qt 6 (Core、Widgets、OpenGL、Sql、Xml、Network)
- **3D 渲染**：OpenSceneGraph 3.4/3.7、OpenGL、GLEW
- **数学库**：Eigen 3 (线性代数)、Ceres Solver (非线性优化/捆绑调整)
- **地理空间**：GDAL、PROJ、GeographicLib、GEOS
- **图像处理**：OpenCV、FreemImage、Exiv2 (EXIF 读取)
- **数据格式**：RapidJSON、pugixml、SQLite3、libxInt (Excel)、FreeXL
- **网络**：libcurl、Qt Network、OpenSSL
- **日志**：glog (Google Logging Library)
- **其他**：Boost (filesystem 等)、Expat (XML 解析)、gflags (命令行参数)、KML 支持

2. 整体架构设计

2.1 三层进程架构



- 执行一个算法步骤后退出 - 写反馈文件（进度、结果）给引擎	
--	--

2.2 为什么采用进程隔离？

这是项目的**核心架构决策**之一：

维度	选择	理由
稳定性	进程隔离而非线程	Task 崩溃不影响 Engine，"崩溃即清理"
资源管理	短生命周期进程	OS 自动回收内存/GPU 资源，避免泄漏累积
可扩展性	独立进程	未来可分布到不同机器（虽然当前未实现）
更新灵活性	独立可执行文件	Engine 和 Task 可独立升级
调试简单	文件 IPC	任务状态以文件形式可见，无需抓包/调试共享内存

2.3 两种任务类型对比

特性	传统 AT/重建任务	生成式 AI 任务
执行方式	fork MoldiaTask.exe 子进程	HTTP 调用远程 AI 服务器
状态跟踪	子进程退出码 + 反馈文件	HTTP 轮询 <code>/generation/getTaskStatus</code>
进度报告	子进程写 <code>JF_*.bin</code> 反馈文件	服务端返回 progress 百分比
积分结算	任务完成后按实际消耗结算	冻结时预估 + 完成后服务端返回结算
文件格式	<code>T_*.json</code> 或 <code>T_*.bin</code>	<code>J_*.bin</code> (XOR 0xAB 加密)
队列目录	<code>jobs/Pending/</code>	<code>jobs_gen/Pending/</code>

3. 核心模块详解

3.1 引擎调度层 (App/Engine/)

CallEngine.cpp (~5700 行) ——引擎主循环

这是**代码量最大的单文件**，包含引擎的完整传统任务生命周期：

启动流程（8 个阶段）：

- 环境准备：控制台编码、SEH 异常处理器、Qt 初始化
- 引擎身份注册：版本号、工作路径、机器码
- 配置解析：读取 `MoldiaConfig.ini`
- 互斥检查： `CatchProcess::NumProgramRunning` 确保单实例

- 5. 信号注册与退出处理器
- 6. 目录创建与残留锁文件清理
- 7. 引擎注册文件准备
- 8. 后台线程启动

启动的后台线程（共 7 个）：

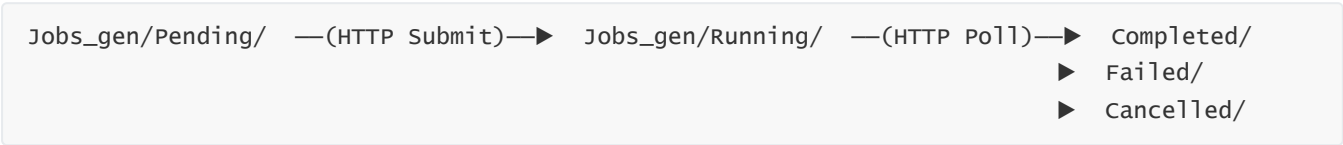
线程函数	职责	轮询间隔
<code>GenTaskThread::Run()</code>	生成式任务调度	2s（待处理） / 30s（运行中）
<code>GenTaskThread::SearchUnnormalRunningJob()</code>	生成式任务看门狗	检测卡死超 24h 的任务
<code>searchPendingJobThread2</code>	传统任务轮询	扫描 <code>jobs/Pending/</code>
<code>searchUnnormaldRunningJobThread</code>	传统任务看门狗	检测异常运行中的任务
<code>searchSettleRetryThread</code>	积分结算重试	1 小时
<code>execTaskTimeThread</code>	任务计时	记录执行耗时
<code>DoCleanupLockFiles</code>	锁文件清理	清理残留 <code>.lock</code> 文件

关键设计点：

- 所有线程通过 `bquitingApplication` 全局标志进行优雅关闭
- 不设线程池——每个线程有固定角色，通过文件系统状态协调

GenTaskThread——生成式任务调度器

纯静态方法类，负责 AI 生成式 3D 任务的完整生命周期：



ProcessPendingJobs 流程：

1. 扫描 `J_*.bin` 文件
2. 反序列化作业信息（含 deny-write 锁）
3. 崩溃恢复检测（检查是否有残留的冻结积分）
4. 上传本地文件（如图片）到服务端
5. HTTP POST 提交任务 → 获得 `freeze_no`
6. 调用 `/point/freeze` 冻结积分
7. 回填 `freeze_no`，保存作业
8. 在 Block 文件中注册作业
9. 原子性移动到 Running 目录

ProcessRunningJobs 流程：

1. 扫描 Running 目录的 `J_*.bin`
2. HTTP GET 查询服务端状态
3. 根据返回的 `GenTaskStatus` 枚举分派：
 - `SUCCESS` → 下载结果文件 → 移动到 Completed → 解冻/结算积分
 - `FAILED` → 提取错误信息 → 移动到 Failed
 - `CANCELLED` → 移动到 Cancelled
 - `RUNNING/QUEUED` → 更新进度反馈

看门狗逻辑 (SearchUnnormalRunningJob) :

- 超过 24 小时未完成 → 标记 FAILED
- 超过 1 小时且 `freeze_no` 为空 → 回退到 Pending 重试

3.2 算法执行层 (App/Task/)

MoldAITask.cpp——传统任务执行

架构为函数映射表驱动模式：

```
// 通过 taskName → 函数指针映射
maptaskfunction["RunReconstruction"] = RunReconstruction;
maptaskfunction["RunMatch"]         = RunMatch;
maptaskfunction["RunSfm"]           = RunSfm;
maptaskfunction["ExecBatchPrepare"] = ExecBatchPrepare;
// ... 等
```

执行流程：

1. 解析命令行参数，获得任务定义文件路径
2. 调用 `MyrapidJson::parseFunctionName()` 从 JSON 中提取 `function` 字段
3. 查表获得函数指针，执行
4. 通过 `cbProgress()` 回调将进度写回反馈文件
5. 执行完成后正常退出进程

进度报告机制：

- 使用 `ReconstructCallback` 类作为回调，定期写 `JF_*.bin` 反馈文件
- 引擎端读取反馈文件更新前端 UI
- 反馈数据结构包含：状态枚举、百分比、消息文本、时间戳

3.3 核心算法库 (MoldiaData.dll)

BlockObject——中心领域对象

`BlockObject` 是项目中最重要的数据模型，代表一个摄影测量"块"：

- 包含：图像列表、相机参数、连接点 (tie points)、控制点 (GCP)、重建结果
- 管理：传统任务和生成式任务的元数据、积分信息
- 持久化：支持 XML 和二进制 (`.bbin`) 两种格式
- 全局实例跟踪：`m_setBlockObject` 静态集合管理所有活跃 Block

`Task_Info` 嵌套结构保存所有任务相关数据：

- `generations_info_`：生成式任务的结果列表
- `reconstructions_info_`：传统重建任务的结果列表
- `generationjobs_` / `reconstructionjobs_`：task_uuid → job_name 映射
- `next_generation_id`：自增 ID

3.4 坐标参考系统 (Include/Core/Proj/)

21 个头文件实现了完整的空间参考系统支持：

- 坐标系定义与查找 (EPSG 编码)
- 坐标变换 (地理坐标 ↔ 投影坐标)
- 基准面变换 (Datum Transform)
- PROJ 数据库管理

这是摄影测量领域的基础设施——所有地理定位、GCP 配准、坐标系统一都依赖于此。

4. 关键设计模式与技术决策

4.1 单例模式 (Singleton)

- **Application**：Meyer's Singleton (函数内 static 局部变量)，C++11 起线程安全
- 提供全局配置访问、版本管理、路径解析

4.2 静态方法类 (Stateless Utility Pattern)

- **GenTaskThread**：纯静态方法，无实例状态
- **PointManager**：纯静态方法，积分操作
- **GenHttpClient**：纯静态方法，HTTP API 封装

这种设计使这些组件成为"无状态服务"——所有状态来自参数或文件系统。

4.3 函数映射表 (Command Pattern / Strategy)

在 `MoldAITask.cpp` 中, 使用 `std::map<string, function_ptr>` 将任务名称映射到具体的算法函数, 实现了:

- 可扩展的任务类型注册
- JSON 中的 `function` 字段直接路由到执行代码

4.4 状态机——基于目录的文件系统状态机

这是项目**最独特**的设计:

状态转换 = 文件在不同目录间移动

- Pending 目录 → 任务待处理
- Running 目录 → 任务执行中
- Completed/Failed/Cancelled 目录 → 终态

优势:

- 原子性: 同磁盘的文件移动 (rename) 是原子的
- 可观测性: `ls` 命令就能看到所有任务状态
- 崩溃恢复: 进程重启后扫描各目录即可恢复状态
- 无中心化状态存储: 不依赖内存中的状态表或数据库

4.5 XOR 0xAB 加密——简单混淆

作业文件 (`.bin`) 使用 XOR 0xAB 进行简单加密:

- 目的: 防止用户直接修改作业数据 (非安全加密)
- 实现: 逐字节 XOR 0xAB
- 这不是真正的加密——只是防止随手编辑的混淆

5. 进程间通信与并发模型

5.1 IPC 方案: 纯文件系统

项目**不使用**任何传统的 IPC 机制 (管道、Socket、共享内存), 完全依赖文件系统:

通信方向	机制	文件格式
前端 → 引擎	任务文件写入 Pending 目录	<code>J_*.bin</code> / <code>T_*.json</code>
引擎 → Task	命令行参数 (启动子进程)	任务文件路径
Task → 引擎	反馈文件写入	<code>JF_*.bin</code>
引擎 → 前端	引擎注册文件 + 任务状态目录	<code>engine_reg.bin</code>

5.2 互斥锁——Deny-Write 文件锁

并发的核心原语是 Windows 的 **deny-write 文件锁**：

```
// File::FopenDenyWriteLockUtf8 的实现原理
FILE* fp = _fsopen(path, "w", _SH_DENYWR);
// _SH_DENYWR：禁止其他进程以写入模式打开此文件
// 关闭文件即释放锁
fclose(fp);
```

锁协议：

- 1. 获取 `.lock` 文件的 deny-write 锁
- 2. 如果获取失败，等待 2 秒后重试（最多 3 次）
- 3. 锁内执行读写操作
- 4. 关闭锁文件释放

这本质上是**自旋锁 + 文件系统的协同咨询锁**——所有参与方必须自觉获取锁。

5.3 线程模型

- **固定角色线程**：每个线程有明确、不变的职责
- **无互斥锁/条件变量**：线程间通过文件系统协调而非共享内存
- **全局退出标志**：`bQuittingApplication` 是唯一的共享状态
- **无工作窃取/动态调度**：线程创建时角色已确定

5.4 并发安全性分析

优势：

- 死锁风险低：文件锁有超时，目录状态可恢复
- 进程隔离天然安全

劣势：

- 文件 I/O 开销大于内存同步
- 轮询间隔导致响应延迟（最坏 30 秒）
- 文件残留可能导致状态不一致

6. HTTP 通信与 API 集成

6.1 双层 HTTP 客户端

层	类	职责
底层	<code>HttpClient</code>	Qt Network 封装、HMAC 签名、同步 POST/GET
业务层	<code>GenHttpClient</code>	生成式任务 API、积分 API

6.2 HMAC-MD5 认证签名

`HttpClient::calSign()` 实现了自定义的 API 签名方案：

```
签名原文 = "moldai:" + path + ":" + timestamp + ":" + JSON(params) + ":" + accessToken
签名值    = Base64(MD5(签名原文))
Authorization = "timestamp:{ts},sign:{sig},accessToken:{tok}"
```

这是一个基于 **HMAC 模式**的请求签名，用于防止请求篡改和重放攻击：

- 时间戳参与签名 → 防重放（服务端校验时间窗口）
- 参数参与签名 → 防参数篡改
- AccessToken 参与签名 → 身份绑定

6.3 生成式任务 API 调用

API	方法	用途
<code>/generation/create3DTask</code>	POST	提交任务（参数含 prompt、polygon_count 等）
<code>/generation/getTaskStatus</code>	GET	轮询任务状态
<code>/point/estimate</code>	POST	积分预估
<code>/point/freeze</code>	POST	积分冻结
<code>/point/settle</code>	POST	积分结算

6.4 同步 HTTP 的 EventLoop 模式

`HttpClient::post()` 使用**同步风格 API 包装异步 Qt Network**：

```
QNetworkReply* reply = nam.post(request, byteArray);
QEventLoop loop;
connect(reply, &QNetworkReply::finished, &loop, &QEventLoop::quit);
loop.exec(); // 阻塞直到请求完成
```

这种模式的**优势**是代码线性易读，**劣势**是调用线程被阻塞——在后台线程中调用是可以接受的。

6.5 网络错误处理

- 超时：通过 `QNetworkRequest` 设置（默认 1000ms）
 - 重试：通过 `max_retries` 参数控制（HTTP 客户端层）
 - 查询重试计数：`query_retry_count`（业务层），连续 5 次超时标记为 FAILED
 - Mock 模式：`MOCK_GEN_HTTP` 宏为开发/测试提供模拟响应
-

7. 积分/计费系统

7.1 数据结构

```
struct PointInfoBase {
    std::string freeze_no;           // 冻结单号（全局唯一）
    int frozen_points;               // 已冻结积分
    int consumed;                   // 实际消耗
    int refunded;                   // 已退款
    int points_settled;              // 是否已结算
    int total_balance;              // 总余额
    int available_points;           // 可用余额
};
```

7.2 积分生命周期

- 1. 预估 (Estimate) → 任务创建前，预估所需积分
- 2. 冻结 (Freeze) → 任务提交后，冻结预估积分（不可用于其他任务）
- 3. 结算 (Settle) → 任务完成后，按实际消耗多退少补

7.3 结算失败兜底机制

这是项目的一个**重要容错设计**：

- 结算失败时，任务信息被写入 `jobs_settle_retry/Pending/` 目录
- `searchSettleRetryThread` 线程每小时轮询一次重试
- 这保证了即使临时网络故障，积分也不会丢失

7.4 两阶段积分模型

阶段	操作	发生时机
冻结	<code>postPointsFreeze()</code>	任务提交到服务端后
结算	<code>postPointsSettle()</code>	任务完成/失败/取消

冻结确保用户不能同时用同一笔积分创建多个任务。

8. 数据序列化策略

8.1 三种序列化格式并存

格式	用途	库
BIN (XOR 0xAB)	作业文件持久化	自定义
JSON (RapidJSON)	API 通信、配置、Block 元数据	RapidJSON

格式	用途	库
XML (pugixml)	Block 数据、AT 数据、配置	pugixml

8.2 BIN 序列化设计

```
// DataStruct.h 中 GenJobFile 的结构
struct GenJobFile {
    GenJobInfoData genJobInfoData;    // project_path, block_item
    GenJobTaskData genJobTaskData;    // 所有任务字段 (params 以 JSON 字符串嵌入)
    JobFeedBackData feedBackData;    // status, percent, msg

    bool Serialize(FILE* fp);        // 写入 BIN
    bool Deserialize(FILE* fp);      // 读取 BIN
};
```

特点:

- `params` (GenTaskParams) 先序列化为 JSON 字符串, 再嵌入 BIN 流
- JSON 在 BIN 中: 这种 BIN+JSON 的混合模式在嵌入式/游戏行业常见
- endian-aware: `ReadBinaryBlob/WriteBinaryBlob` 处理字节序

8.3 JSON 序列化

- 全手动: 没有代码生成器或反射框架
- 每个结构体手写 `WriteToJson()` / `ParseJson()` 方法
- `std::optional` 用于可选字段的语义建模 (如 `GenTaskResponse`)

8.4 序列化迁移状态

项目正在进行从 XML → JSON 的渐进迁移:

- Block 信息同时支持 `ReadBlockInfoBin/Json` 和 `WriteBlockInfoToBin/Json`
- 这是典型的**绞杀者模式 (Strangler Fig)** —— 新旧并存直到旧格式完全淘汰

9. 摄影测量算法管线

9.1 AT (空中三角测量) 流程

```
图像输入 → 特征检测(SIFT等) → 特征匹配 → 运动恢复结构(SfM)
→ 全局捆绑调整(BA/ Ceres) → 稀疏点云输出
```

关键算法模块:

- **特征检测:** `AlgorithmBase.h` 中定义的接口
- **RANSAC:** `Loransac.h`、`RandomSampler.h` —— 用于鲁棒估计
- **捆绑调整:** 使用 Ceres Solver 进行非线性最小二乘优化

- **相似变换**: `SimilarityTransform.h`、`alignment.h` ——用于坐标系对齐

9.2 三维重建

- 从 AT 输出的稀疏点云生成稠密模型
- 支持高斯泼溅 (Gaussian Splatting) ——通过 `isGDGSEnable()` / `isBaseGSEnable()` 特性开关
- 网格生成、纹理映射

9.3 相机模型

- 支持多种相机模型 (针孔、鱼眼等)
- 相机数据库 (`CameraDatabase.h`) 管理已知相机参数
- EXIF 读取 (`ExifIO.h`) 自动提取焦距、传感器尺寸

10. 生成式 AI 任务集成

10.1 支持的 AI 任务类型

```
enum class GenTaskSubType {  
    TEXT_TO_MODEL,           // 文本→3D模型  
    TEXT_TO_MESH,           // 文本→网格  
    IMAGE_TO_MODEL,          // 图像→3D模型  
    TEXT_TO_TEXTURE,         // 文本→纹理  
    // ... 等 10 种  
};
```

10.2 任务参数结构

```
struct GenTaskParams {  
    std::string prompt;           // 文本提示 (正面)  
    std::string negative_prompt;  // 负面提示  
    int polygon_limit;            // 多边形面数限制  
    int texture_size;            // 纹理分辨率  
    int provider_id;             // AI 服务提供商  
    std::string model_version;    // 模型版本  
    std::string image_file;       // 输入图像 (Image-to-3D 模式)  
};
```

10.3 分块任务失败兜底

- 大任务被拆分为多个子任务 (分块)
 - 如果某个分块失败, 引擎会自动重试或跳过该分块
 - 兜底策略确保"部分成功"优于"全部失败"
-

11. 构建系统与依赖管理

11.1 CMake 架构

- 根 `CMakeLists.txt` 定义项目 `MoIdia` (C++17)
- 自定义 CMake 模块系统:
 - `Prequisite.cmake`: 预置条件检查
 - `CMakeExternalLibs.cmake`: 外部库管理
 - `Ai3dMsvcUtf8.cmake`: UTF-8 编译配置
- `CMakePresets.json`: VS2022/MSVC 预设

11.2 依赖库清单 (20+ 个)

类别	库	用途
数学	Eigen3, Ceres	线性代数、非线性优化
GIS	GDAL, PROJ, GEOS, GeographicLib, KML	坐标系统、地理数据
图像	OpenCV, FreeImage, Exiv2	图像处理、EXIF
3D	OpenSceneGraph, GLEW, VCG	3D 渲染、网格处理
网络	libcurl, OpenSSL	HTTP、加密
数据	RapidJSON, pugixml, SQLite3, libxInt	JSON/XML/SQL/Excel
框架	Qt6, Boost	UI、工具
日志	glog, gflags	日志、命令行

11.3 依赖管理的挑战

- 20+ 个 C++ 库的组合：版本兼容性是一个持续的挑战
- 静态链接 vs 动态链接的权衡
- DLL 导入/导出宏 (`AI3D_API`、`AI3DEXPORT`) 的跨平台配置

12. 容错与异常处理

12.1 多层容错策略

层级	机制	说明
进程级	SEH 异常处理 + 信号处理	防止进程崩溃
任务级	进程隔离	Task 崩溃不影响 Engine
文件级	Deny-write 锁 + 重试 (3次)	防止并发写冲突

层级	机制	说明
网络级	超时 + 重试 + 退避	HTTP 请求容错
业务级	看门狗线程	检测卡死任务、恢复到健康状态
结算级	重试队列	结算失败后每小时重试

12.2 错误码体系

项目有**两套独立的错误码**（这是代码质量问题）：

- **ReturnState.h**：100000-300012 范围，约 30 个码，覆盖算法级错误
- **ReturnCode.h**：1000-30001 范围，约 15 个码，覆盖文件/图像/请求错误

12.3 崩溃恢复

- **启动时清理**：DoCleanupLockFiles 清理上次未正常退出的残留锁文件
- **作业恢复**：启动后扫描 Running 目录，将异常任务重新诊断
- **冻结积分恢复**：检测 freeze_no 为空但有冻结积分的异常状态
- **引擎注册文件**：用于检测引擎是否正常运行

13. 启动诊断机制

13.1 BootProbe——静态初始化诊断

这是一个巧妙的诊断工具，用于定位启动阶段崩溃：

```
// 在 main() 之前运行（静态初始化器）
struct BootProbeStaticInitializer {
    BootProbeStaticInitializer() {
        WriteBootProbeLine("before_main_static_ctor");
    }
};

static BootProbeStaticInitializer g_bootProbeInit;

// main() 入口第一行
void EngineBootProbeMainEnter() {
    WriteBootProbeLine("main_enter");
}
```

13.2 设计精妙之处

- **不依赖任何初始化好的全局状态**：使用原生 Win32 API（CreateFileA/writeFile）而非 C 运行时
- **时间戳精确到毫秒**：帮助定位性能瓶颈
- **OutputDebugStringA 同步输出**：VS 调试器可实时看到
- **带版本标记**：BOOT_PROBE_MARKER_V2_20260324 用于日志解析兼容性

这回答了"如果程序在 main 之前就崩溃了，如何知道"的经典问题。

14. 代码质量与改进方向

14.1 当前优点

- ✅ 进程隔离架构：稳定性和可维护性的坚实基础
- ✅ 文件系统状态机：简单、可观测、崩溃友好
- ✅ 容错设计完善：多层重试、看门狗、兜底策略
- ✅ 诊断机制完备：BootProbe、日志分级、版本标记
- ✅ 两阶段积分模型：冻结-结算保证财务安全

14.2 可改进之处

问题	影响	建议
两套错误码体系	混乱、转换易出错	统一为一个 <code>std::error_code</code> 体系
<code>CallEngine.cpp</code> ~5700行	维护困难、难以测试	拆分为多个职责清晰的类
文件锁手动管理	RAII 缺失 → 潜在泄漏	封装 <code>LockFileGuard</code> RAII 类
缺少单元测试覆盖	回归风险	关键路径添加 <code>gtest</code>
<code>bQuittingApplication</code> 全局变量	隐式依赖	改为显式 <code>token/stop_source</code>
硬编码魔法数字	可读性差	提取为命名常量
拼写错误 (PENDDING/CANCLE)	接口一致性	修正或建立别名
无自动化 CI/CD 检查	质量无保障	GitHub Actions lint + build
BIN 序列化全手写	易出错	考虑 <code>protobuf/flatbuffers</code>

15. 面试高频问题 Q&A

Q1: "这个项目是做什么的？"

这是一个 **Windows 桌面端的摄影测量与 3D 重建引擎**。简单说，它能从大量照片（比如无人机航拍）自动生成三维模型。核心能力包括：空中三角测量（从照片恢复相机位置和稀疏点云）、三维重建（生成稠密模型和纹理）、以及最新的 AI 生成 3D 模型（输入文字描述或图像，远程 AI 服务生成 3D 资产）。还包含一个积分计费系统来管理这些计算任务的费用。

Q2: "你在项目中负责什么？"

(根据你的实际情况调整) 例如: 负责引擎调度层的设计与开发, 包括任务队列系统、进程调度、HTTP 通信层、积分系统集成、以及各种容错机制(看门狗、崩溃恢复、结算重试)。

Q3: "项目的架构是什么样的? 为什么这样设计?"

采用**三层进程架构**: 前端 UI → 引擎调度进程 (MoldiaNode) → 算法执行进程 (MoldiaTask)。核心设计思想是**进程隔离**: 把调度和算法执行分到不同进程, Task 崩溃不影响 Engine, OS 自动回收资源。进程间通过**文件系统 IPC** 通信——任务文件在不同目录间移动代表状态流转。这种方法简单、可观测 (1s 就能看状态)、崩溃友好 (重启后扫描目录即可恢复)。

Q4: "如何保证任务不会重复执行或丢失?"

核心是 **Windows deny-write 文件锁 + 目录化状态机**。每个任务文件在被处理前需要获取 `.lock` 文件的排他写锁 (`_SH_DENYWR`)——同一时刻只有一个线程/进程能持有。任务状态通过目录迁移表示 (Pending → Running → Completed), `rename` 在同磁盘上是原子操作。即使进程崩溃, 重启后扫描各目录即可确定每个任务的最新状态。

Q5: "HTTP 请求的安全性如何保证?"

实现了自定义的 **HMAC-MD5 请求签名** 方案。签名原文包含: 路径、时间戳、请求参数 JSON、AccessToken。签名为 `Base64(MD5(原文))`。服务端通过校验时间戳窗口防重放攻击, 通过校验签名防参数篡改, 通过 AccessToken 验证身份。

Q6: "如何处理网络不稳定导致的请求失败?"

多层容错: HTTP 层有超时和重试机制; 业务层有 `query_retry_count` 计数器 (连续 5 次失败才标记任务失败); 积分结算有专门的**重试队列**——失败的结算请求写入 `jobs_settle_retry/` 目录, 独立线程每小时轮询重试。这是典型的**最终一致性**设计。

Q7: "积分的冻结和结算是什么? 为什么需要两步?"

冻结是在任务提交时**预留**积分 (防止用户同时用同一笔积分创建多个任务), 结算是在任务完成后根据**实际消耗**多退少补。这类似于酒店预订授权——入住时冻结押金, 退房时按实际消费结算。

Q8: "为什么用文件系统做 IPC 而不是消息队列或 gRPC?"

权衡: 文件 IPC 性能不如消息队列, 但有独特优势——状态天然持久化 (崩溃不丢)、极强的可观测性 (`ls / cat` 即可调试)、无额外依赖、同磁盘 `rename` 是原子的。对于这个项目的吞吐量需求 (秒级任务调度), 文件 I/O 的开销完全可以接受。

Q9: "如果让你重新设计一个模块, 你会怎么做?"

我会把 `CallEngine.cpp` (5700 行) 重构为多个职责清晰的类: `TaskScheduler` (调度)、`TaskExecutor` (执行)、`JobLifecycleManager` (生命周期)、`FeedbackCollector` (反馈收集)。引入 RAII 封装文件锁。统一两套错误码为一个 `std::error_code` 体系。为关键路径 (任务状态机、积分结算) 补充单元测试。

Q10: "项目中用了哪些 C++17 特性？"

- `std::optional`：HTTP 响应中的可选字段（`GenTaskResponse`）
- `std::filesystem`：路径操作（UTF-8 → path 转换、目录遍历）
- `if constexpr` / fold expressions： `JoinPaths` 变参模板
- Structured bindings：部分数据解构
- Meyer's Singleton： `Application::GetInstance()` 线程安全初始化

Q11: "BootProbe 是什么？为什么需要它？"

BootProbe 是**启动阶段诊断工具**。它利用 C++ 静态初始化顺序，在 `main()` 之前写入时间戳日志。如果程序在启动时崩溃，通过日志可以判断是"全局变量初始化阶段崩溃"还是"main 之后崩溃"。它使用原生 Win32 API（而非 C 运行时），因为此时 C 运行时可能还未初始化。

Q12: "项目中最大的技术挑战是什么？"

（结合你实际经历）可以从几个角度回答：

1. **进程生命周期管理**：确保 Task 进程在各种异常情况下（崩溃、卡死、被强杀）都能被正确回收
2. **并发文件操作**：多线程同时读写同一目录的文件，deny-write 锁的竞态条件处理
3. **积分一致性**：冻结-结算的两阶段提交模式在分布式（客户端-服务端）环境下的最终一致性保证
4. **大文件处理**：摄影测量项目可能包含数百 GB 的图像数据，内存管理和 I/O 优化

附录：项目文件速查

关键源文件

文件	行数	角色
<code>App/Engine/CallEngine.cpp</code>	~5700	引擎主循环、传统任务调度
<code>App/Engine/GenTaskThread.cpp</code>	~400	生成式任务调度
<code>App/Engine/BootProbe.cpp</code>	~60	启动诊断
<code>App/Task/MoldAITask.cpp</code>	~900	任务执行入口
<code>App/Task/ATPreprocessTask.cpp</code>	~100	AT 预处理
<code>Src/Core/BlockObject.cpp</code>	~12000	核心域对象
<code>Src/Core/GenHttpClient.cpp</code>	~350	AI 任务 HTTP API
<code>Src/Util/HttpClient.cpp</code>	~370	通用 HTTP 客户端 + HMAC

关键头文件

文件	内容
Include/Core/BlockObject.h	中心域对象定义
Include/Core/GenTaskOptions.h	生成式任务状态、参数
Include/Core/GenHttpClient.h	AI API 接口
Include/Core/PointManager.h	积分系统接口
Include/Core/Application.h	应用单例
Include/Core/Constants.h	DLL 宏、类型定义
Include/Core/File.h	文件操作 + deny-write 锁
Include/Core/ReturnState.h	错误码（算法级）
Include/Core/ReturnCode.h	错误码（通用级）
Include/Util/GenTaskProcess.h	生成式任务数据结构
Include/Util/TaskProcess.h	传统任务数据结构
Include/Util/HttpClient.h	HTTP 客户端定义
Include/Core/DataStruct.h	BIN 序列化结构

设计文档

文件	大小	主题
App/doc/详细分析文档.md	64KB	8 章全面分析
App/doc/GenTask_Checklist.md	199KB	分阶段执行清单
App/doc/积分接口集成方案.md	86KB	积分系统设计
App/doc/积分系统测试用例.md	58KB	积分测试用例
App/doc/任务终态结算位置分析.md	30KB	结算最终一致性分析
App/doc/CancelJob_Running残留问题分析.md	7KB	任务取消边界情况

使用建议：面试前重点复习第 2-7 节（架构、IPC、HTTP、积分），以及第 15 节的 Q&A。对于你具体负责的模块，准备 2-3 个深入的技术细节案例（如"你是怎么解决 XX 问题的"）。